L2M: An LLM-Driven Lyrics-to-Melody Generation System with Emotion-Aware Alignment

Submitted to the

Department of Computer Science and Engineering,
Mawlana Bhashani Science and Technology University
in partial fulfillment of the requirements for the degree of
M.Engg. in CSE.

Submitted by

Md Arafat Hasan Jenin, *(CE-210926)*

Under the Supervision of

Professor Dr. Mohammad Motiur Rahman
Department of Computer Science and Engineering

Department of Computer Science and Engineering
Mawlana Bhashani Science and Technology University
Santosh, Tangail-1902, Bangladesh

August, 2026

L2M: An LLM-Driven Lyrics-to-Melody Generation System with Emotion-Aware Alignment

Submitted by
Md Arafat Hasan Jenin, *(CE-210926)*

Under the Supervision of
Professor Dr. Mohammad Motiur Rahman
Department of Computer Science and Engineering

Submitted to the
Department of Computer Science and Engineering,
Mawlana Bhashani Science and Technology University
in partial fulfillment of the requirements for the degree of
M.Engg. in CSE.

Project Evaluation Committee:

Teacher Name 1 .....

Teacher Name 2 .....

Teacher Name 3 .....

Project Approval

Md Arafat Hasan Jenin, *(CE-210926)*

L2M: An LLM-Driven Lyrics-to-Melody Generation System with Emotion-Aware Alignment

I the undersigned, recommend that the project completed by the student listed above, in partial fulfillment of M.Engg. in CSE degree requirements, be accepted by the Department of Computer Science and Engineering, Mawlana Bhashani Science and Technology University for deposit.

Supervisor Approval*

.....

Professor Dr. Mohammad Motiur Rahman
Department of Computer Science and Engineering

Departmental Approval*

.....

Chairman
Department of Computer Science and Engineering

Mawlana Bhashani Science and Technology University
Santosh, Tangail-1902, Bangladesh

Declaration

I hereby declare that this project report titled, ***"L2M: An LLM-Driven Lyrics-to-Melody Generation System with Emotion-Aware Alignment"*** is my own research work and I confirm that any part of this project report has not been submitted yet for a degree or any other qualification at this university or any other institution.

.....

Md Arafat Hasan Jenin

Student ID: CE-210926

dedicated to my
beloved parents

Abstract

Generating a singable melody from lyrical text is a demanding cross-modal task: the system must understand the semantic and emotional content of the lyrics, align every syllable with exactly one musical note, and produce an output that is musically coherent and emotionally consistent with the text. Existing approaches rely on supervised deep learning models trained on large paired lyrics–melody corpora, which are difficult and expensive to curate, and they typically fail without recourse when the model produces invalid output at inference time. This project presents **L2M**, a lyrics-to-melody generation system that employs a Large Language Model (LLM) as its core reasoning engine in a zero-shot setting, requiring no task-specific training. L2M decomposes the task into a two-stage prompting pipeline: the first LLM call classifies the emotional content, tempo, and phrase structure of the input lyrics, and a second call generates a syllable-aligned note sequence conditioned on the extracted musical attributes. An explicit emotion-to-key mapping grounds the generation in Western music theory, while a deterministic fallback mechanism based on melodic contour heuristics guarantees output regardless of API availability. The system exports standard music notation (MIDI, MusicXML) and optional synthesized audio (WAV/MP3). Evaluation on a 20-sample benchmark spanning six emotion categories shows 100% syllable–note alignment accuracy, 95% emotion–key consistency, 100% MIDI validity, and an average end-to-end latency of 3.2 seconds. L2M is released as an open-source Python package with a command-line interface and programmatic API, providing an accessible and reproducible baseline for LLM-based music generation research.

Keywords: Lyrics-to-Melody Generation, Large Language Models, Prompt Engineering, Emotion-Aware Music Generation, Music Information Retrieval, Cross-Modal Generation.

Acknowledgment

First and foremost, at this very special moment, I would like to express my sincere gratitude to the Almighty Allah for helping me to complete this degree. I am very grateful not only throughout my study time, but also during my life, for the tremendous blessings the Almighty has conferred upon me.

I would like to express my deepest gratitude to my supervisor, Professor Dr. Mohammad Motiur Rahman, Department of Computer Science and Engineering, Mawlana Bhashani Science and Technology University, for his continuous guidance, insightful feedback, and encouragement throughout the course of this research. His advice on both the technical and academic aspects of this work has been invaluable.

I am also thankful to the faculty members of the Department of Computer Science and Engineering for the knowledge and support they have provided during my studies, and to my family and friends for their patience and constant encouragement.

Md Arafat Hasan Jenin
August, 2026

Contents

List of Figures

List of Tables

Chapter 1
Introduction

Chapter 1

Introduction

Music has accompanied human language for as long as recorded history, and the song — lyrics set to melody — remains one of the most universal forms of creative expression. Composing a melody for a given lyric, however, is a task that has traditionally demanded years of musical training. It requires the composer to understand what the words *mean*, to feel what they *evoke*, and to translate both into a sequence of pitches and durations that a human voice can sing naturally. This project investigates whether a Large Language Model (LLM), used purely through prompting and without any task-specific training, can perform this act of cross-modal composition, and presents **L2M**, a complete, open-source lyrics-to-melody generation system built on this premise.

1.1 Motivation

Automatic music composition has advanced rapidly with deep learning, and systems now exist that generate convincing instrumental music [1], [2] and even high-fidelity audio directly from text descriptions [3], [4]. Yet the specific task of *lyrics-to-melody generation* — producing a singable melodic line for a given lyrical text — remains comparatively difficult, because it couples natural language understanding with symbolic music generation under hard structural constraints.

Existing lyrics-to-melody systems [5]–[9] are almost exclusively supervised: they learn the mapping from lyrics to melody from large corpora of paired examples. Such corpora are scarce, expensive to curate, and often encumbered by copyright, which restricts both research reproducibility and practical deployment. Moreover, purely neural systems typically offer no recourse when the model produces an invalid output at inference time — a wrong number of notes, an out-of-range pitch, or a malformed sequence simply becomes a failed run.

The emergence of instruction-following LLMs trained on internet-scale corpora — which include music theory texts, song analyses, and transcribed lyrics — suggests an alternative path. If an LLM already encodes substantial implicit

knowledge about the relationship between words, emotions, and music, then careful *prompt engineering* may elicit that knowledge zero-shot [10], eliminating the need for paired training data altogether. Whether this is actually achievable, and how reliably, is the central question motivating this work.

1.2 Problem Statement

The lyrics-to-melody task imposes four tightly coupled requirements on any generation system:

1. **Semantic understanding.** The system must interpret not merely the denotative meaning of the words but the affective intent and thematic atmosphere embedded in the text.
2. **Syllabic alignment.** Every phonetic syllable must correspond to exactly one musical note, preserving the natural cadence of the sung phrase and ensuring singability.
3. **Musical coherence.** The output must exhibit a well-chosen key, consistent tempo, appropriate mode, and a melodic contour that forms a satisfying phrase arc within the conventions of tonal music.
4. **Emotional consistency.** The affective character of the melody must reinforce, rather than contradict, the sentiment expressed in the lyrics.

Formally, given an arbitrary lyrical text L containing N syllables, the system must produce a melody $M = \langle (p_1, d_1), \ldots, (p_N, d_N) \rangle$ — a sequence of exactly N (pitch, duration) pairs — together with a key, tempo, and time signature, such that M is singable, tonally coherent, and emotionally consistent with L. The system must produce a valid M for *every* input, including when the underlying LLM service is unavailable or returns malformed output.

1.3 Research Questions

This project addresses the following research questions:

RQ1. Can a general-purpose LLM, accessed zero-shot through structured prompts, generate melodies that satisfy exact syllable–note alignment and basic tonal coherence, without any task-specific training?

RQ2. Does decomposing the task into two stages — emotion/rhythm analysis followed by conditioned melody generation — improve controllability and interpretability over a single monolithic generation step?

RQ3. Can a deterministic, music-theory-grounded fallback mechanism guarantee full operational reliability while preserving acceptable musical quality when the LLM path fails?

RQ4. Which prompt engineering techniques (output schemas, hard constraints, few-shot examples, contextual carryover) are necessary to make LLM output usable in a strictly validated music pipeline?

1.4 Objectives

The concrete objectives of this work are to:

1. Design an end-to-end pipeline that accepts arbitrary lyrical text and produces standard music notation (MIDI, MusicXML) and optional synthesized audio (WAV/MP3);
2. Develop a two-stage LLM prompting strategy for emotion analysis and syllable-aligned melody generation, with structured JSON output schemas validated at every stage;
3. Ground the generation process in music theory through an explicit mapping from emotion categories to musical keys, tempo ranges, and melodic contour patterns;
4. Implement deterministic fallback heuristics that guarantee a valid output whenever an LLM call fails, times out, or returns malformed data;
5. Evaluate the system quantitatively and qualitatively on a benchmark spanning six emotion categories, and release it as a documented, open-source Python package.

1.5 Contributions

The principal contributions of this project are:

1. **Two-stage LLM pipeline.** A novel decomposition of the lyrics-to-melody task into (a) emotion and rhythm analysis and (b) conditioned melody generation, each handled by a separate LLM call with a structured JSON output schema — enabling interpretability and targeted fallback at each stage.

2. **Emotion-aware alignment.** An explicit mapping from detected emotion categories to musical keys, tempo ranges, and melodic contour patterns, grounding LLM-generated output in established music-psychology findings [11], [12].

3. **Hybrid reliability architecture.** A combination of zero-shot LLM generation with deterministic heuristic fallbacks that achieves a 100% completion rate across all tested inputs.

4. **Prompt engineering methodology.** Structured few-shot prompts with explicit output schemas, hard constraint enforcement ("EXACTLY N notes for N syllables"), and contextual carryover across long-lyric chunks — a reusable template for other cross-modal LLM tasks.

5. **Open-source baseline.** A production-ready Python package offering a CLI, a programmatic API, multi-format output, and comprehensive logging, providing a reproducible baseline for the research community.

1.6 Scope and Limitations

The system generates *monophonic* vocal melodies within the conventions of Western tonal music; harmony, accompaniment, and non-Western musical systems are outside the present scope and are discussed as future work. Evaluation is conducted on a 20-sample proof-of-concept benchmark with automatic metrics and expert qualitative review; a large-scale listener study is likewise deferred to future work.

1.7 Report Organization

The remainder of this report is organized as follows. Chapter 2 surveys prior work on lyrics-to-melody generation, text-to-music synthesis, and LLM-based generation. Chapter 3 introduces the background concepts this work builds upon: transformer-based LLMs, prompt engineering, the psychology of musical emotion, and symbolic music representations. Chapter 4 presents the proposed

six-stage L2M architecture in detail, including both LLM prompt designs and the deterministic fallback algorithms. Chapter 5 reports the experimental setup, quantitative results, qualitative analyses, and a comparison with related systems, followed by a discussion of findings and limitations. Chapter 6 concludes and outlines directions for future research.

Chapter 2
Literature Review

Chapter 2

Literature Review

This chapter reviews the research context in which L2M is situated. The review is organized in four parts: general deep-learning approaches to music generation, supervised lyrics-to-melody systems, text-conditioned audio generation, and the recent application of LLMs to musical tasks. The chapter closes with a gap analysis that motivates the design of L2M.

2.1 Deep Learning for Music Generation

Deep learning approaches to symbolic music generation have employed recurrent networks, GANs, VAEs, and Transformer architectures to model the hierarchical temporal structure of music; Briot et al. [1] provide a comprehensive survey of these foundational methods. Sequence-to-sequence learning [13] first made it practical to map one symbolic sequence to another of different length, forming the algorithmic basis for lyric-conditioned melody generation. The Music Transformer [2] subsequently demonstrated that self-attention with relative position representations can capture long-range musical structure such as repetition and motif development, establishing the Transformer [14] as the dominant architecture for symbolic music modeling.

2.2 Lyrics-to-Melody Generation

2.2.1 Early Sequence-to-Sequence Approaches

Bao et al. [5] introduced one of the earliest end-to-end neural frameworks for melody composition from lyrics, using syllable-level alignment on pop music corpora within a Seq2Seq architecture. The work demonstrated feasibility but required a large paired lyrics–melody corpus, and generated melodies frequently violated basic music-theoretic expectations. SongMASS [6] improved data efficiency by pre-training separate lyric and melody encoders with masked sequence-

to-sequence objectives and enforcing alignment constraints during decoding.

2.2.2 Music-Theory-Constrained and Template-Based Systems

ReLyMe [8] addressed a core weakness of purely data-driven models — weak semantic and musical coherence — by explicitly incorporating lyric–melody relationship principles from music theory (tone, rhythm, and structure relationships) as decoding-time constraints. TeleMelody [7] decomposed generation into a two-stage, template-based process (lyric-to-template and template-to-melody), improving controllability and reducing the dependence on paired data. ROC [15] went further and reformulated generation as retrieval and re-composition over a database of existing melodic fragments. These systems collectively demonstrate the value of decomposition and of explicit musical knowledge — two ideas that L2M adopts in a radically simplified, prompt-based form.

2.2.3 Controllable and Alignment-Focused Systems

Zhang et al. [16] introduced style modulation via reference embeddings and memory fusion, enabling user-directed genre and mood control and marking the shift from pure generation toward interactive music creation. Reddy et al. [17] addressed incomplete or partial lyric inputs by applying cross-attention between text and melody sequences, improving robustness in real-world scenarios. More recent conditional-Transformer systems provide fine-grained melodic control: CSL-L2M [18] conditions on song-level lyric and musical attributes, while SongGLM [19] introduces 2D alignment encoding with multi-task pre-training for harmonized lyric–melody relationships. On the representation side, Wang et al. [20] learn a shared melody–lyrics embedding space with a contrastive alignment loss, demonstrating the importance of strong cross-modal representations for both generation and retrieval.

2.2.4 LLM-Based Song Composition

SongComposer [9] is the closest recent work to this project: a music-specialized LLM that generates lyrics and melodies jointly using a tuple representation for word-level alignment. It achieves high quality but requires extensive domain-specific fine-tuning on large paired song datasets — precisely the requirement L2M eliminates through zero-shot prompting of a general-purpose model. Chat-

Musician [21] similarly extends an open LLM with intrinsic musical abilities via continued pre-training on ABC notation, again at substantial training cost.

2.3 Text-to-Audio Music Generation

A parallel research line generates *audio* rather than symbolic notation. MusicLM [3] casts text-conditioned music generation as hierarchical sequence-to-sequence modeling over audio tokens, and MusicGen [4] showed that a single-stage language model over compressed audio representations suffices for controllable, high-quality generation. These systems produce rich, fully arranged music, but they differ fundamentally from the task addressed here: they do not enforce syllabic alignment with lyrical text, and their audio output cannot be edited as notation by a musician. Recent comprehensive reviews of AI-enabled text-to-music generation [22] classify systems along these symbolic-versus-audio axes and identify persistent challenges: long-term coherence, data scarcity, and limited controllability — challenges that motivate symbolic, interpretable approaches such as L2M.

Related transformer-based work has also targeted the inverse and adjacent tasks, such as lyric text generation with T5-style models [23] and LSTM-based lyric modeling [24], which established sequential baselines for the language side of song composition.

2.4 Gap Analysis

Table 2.1 summarizes the limitations shared by existing lyrics-to-melody systems.

Table 2.1. Gap analysis of existing lyrics-to-melody systems and the corresponding L2M design response.

Limitation in prior work	L2M design response
Require large paired lyrics–melody training corpora	Zero-shot LLM prompting; no training data
No graceful degradation when the model fails at inference	Deterministic, music-theory-grounded fallback at each LLM stage
Single output format (typically MIDI only)	MIDI, MusicXML, WAV, and MP3 export
No explicit emotion-to-key grounding	Explicit emotion-to-musical-attribute mapping
Not available as deployable open-source software	Documented Python package with CLI and API

No prior system, to the best of our knowledge, combines zero-shot LLM generation with a deterministic reliability layer and multi-format notation export in a single deployable package. This combination defines the research space that the remainder of this report develops.

Chapter 3
Background: LLMs and Symbolic Music

Chapter 3

Background: Large Language Models and Symbolic Music

This chapter introduces the concepts on which the proposed system is built: transformer-based Large Language Models and the prompting techniques used to control them, the psychology of musical emotion that grounds the emotion-to-key mapping, and the symbolic music representations used to encode and export generated melodies.

3.1 Large Language Models

Large Language Models are neural networks based on the Transformer architecture [14], trained on internet-scale text corpora with a self-supervised next-token prediction objective. The Transformer replaces recurrence with *self-attention*, in which every token attends to every other token in the context window:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \tag{3.1}$$

where Q, K, and V are query, key, and value projections of the input and d_k is the key dimension. Stacked multi-head attention layers allow the model to capture long-range dependencies — in language and, as this project exploits, in musical structure as well.

Brown et al. [10] demonstrated that sufficiently large models exhibit *in-context learning*: they can perform new tasks specified entirely in the prompt, optionally guided by a small number of examples (*few-shot* prompting), without any parameter updates. This *zero-shot* or few-shot capability is the foundation of L2M. Modern instruction-tuned models such as GPT-4 [25] follow natural language instructions and can emit structured output formats such as JSON on request. Because their pre-training corpora include music theory texts, song analyses, chord charts, and transcribed lyrics, such models implicitly encode a substantial amount of musical knowledge — knowledge that music-specialized LLMs like

ChatMusician [21] make explicit through further training, but which this work instead elicits purely through prompting.

3.2 Prompt Engineering

Prompt engineering is the practice of designing model inputs that reliably elicit a desired behavior. The techniques used in this work are:

- **Role assignment.** A system message assigns the model a persona (e.g., "expert music composer and emotional analyst"), conditioning it toward domain-appropriate output.
- **Output schema specification.** The prompt specifies an exact JSON structure for the response, allowing downstream code to parse and validate the output mechanically.
- **Few-shot examples.** A small number of worked input–output examples [10] demonstrate the expected format and reasoning style; L2M uses three diverse examples per analysis prompt.
- **Hard constraints.** Explicit, non-negotiable requirements (e.g., "generate EXACTLY N notes") that the output must satisfy. Decomposing a task into intermediate reasoning steps is likewise known to improve reliability on structured problems [26]; L2M applies this principle architecturally by splitting analysis from generation.
- **Contextual carryover.** When input must be processed in chunks, appending a summary of previous output (here, the last three generated notes) to each subsequent prompt preserves continuity.

Even with these techniques, LLM output is stochastic and occasionally malformed; every response in L2M is therefore validated against typed schemas before use, and a deterministic fallback stands behind every LLM call.

3.3 Music, Emotion, and Tonality

The link between musical structure and perceived emotion has been studied empirically for nearly a century. Hevner's classic adjective-circle experiments [11] established that listeners reliably associate musical modes with affective qualities — major modes with happiness and serenity, minor modes with sadness and

tension. Russell's circumplex model of affect [27] organizes emotion along two dimensions, *valence* (pleasant–unpleasant) and *arousal* (activated–deactivated), which map naturally onto musical parameters: mode and consonance track valence, while tempo and dynamics track arousal. Subsequent research consolidated these findings into robust structure–emotion associations [12], [28]: fast tempi and ascending contours convey excitement or joy; slow tempi, minor keys, and descending contours convey sadness; moderate tempi with small intervals convey calm.

L2M operationalizes this literature directly: each of its six emotion categories (*happy, sad, hopeful, tense, calm, excited*) is mapped to a key, a tempo range, and a melodic contour pattern (Chapter 4, Table 4.2). Basic tonal-music conventions constrain the output further: melodies remain diatonic to the selected key, within a singable vocal range, and quantized to standard note durations.

3.4 Symbolic Music Representation

3.4.1 Pitch, Duration, and Scientific Pitch Notation

A monophonic melody is a sequence of notes, each defined by a pitch and a duration. Pitches are written in *scientific pitch notation* (SPN), which combines a note letter, optional accidental, and octave number — e.g., C4 is middle C and A4 is the 440 Hz tuning standard. Durations are expressed in quarter-note (beat) units; L2M restricts them to the set $\{0.25, 0.5, 1.0, 2.0, 4.0\}$, i.e., sixteenth note through whole note. Loudness is encoded as MIDI *velocity* (0–127).

3.4.2 MIDI and MusicXML

Two standard interchange formats are used for output. **MIDI** (Musical Instrument Digital Interface) encodes music as timed note-on/note-off events with pitch and velocity; it is universally supported by sequencers and synthesizers. **MusicXML** [29] is an XML-based notation interchange format that preserves score-level semantics — key signatures, time signatures, and note spellings — making the output directly editable in engraving software such as MuseScore or Finale. Exporting both formats serves complementary use cases: playback and production (MIDI) versus reading and editing by musicians (MusicXML).

3.4.3 The music21 Toolkit

music21 [30] is a Python toolkit for computer-aided musicology that provides object models for notes, streams, key and time signatures, and metronome marks, together with validated exporters for MIDI and MusicXML. L2M builds its output stage on music21, which additionally serves as an independent validity check: any generated file that music21 can re-parse without error is structurally well-formed.

3.4.4 Audio Synthesis

Symbolic output is optionally rendered to audio using *FluidSynth*, a SoundFont-based software synthesizer. A SoundFont (`.sf2`) bundles sampled instrument timbres; FluidSynth renders a MIDI file through these samples into PCM audio (WAV), which FFmpeg can further encode to MP3. This completes the path from raw text to listenable audio.

Chapter 4
Proposed Methodology

Chapter 4

Proposed Methodology

This chapter presents the architecture and methodology of the proposed L2M system. The system is organized as a six-stage sequential pipeline in which each stage produces a validated, typed output consumed by the next. Two of the stages invoke an LLM; each of these is paired with a deterministic fallback so that the pipeline always completes.

4.1 Pipeline Overview

Figure 4.1 shows the six-stage pipeline. Raw lyrics are parsed and normalized (Stage 1), analyzed for emotion and rhythm by the first LLM call (Stage 2), and turned into a syllable-aligned note sequence by the second LLM call (Stage 3). The result is converted into a typed internal representation (Stage 4), exported to MIDI and MusicXML (Stage 5), and optionally rendered to audio (Stage 6). At each LLM-dependent stage, a fallback path activates automatically on API failure, timeout, or malformed response.

4.2 Design Principles

Five principles guided the system design; Table 4.1 summarizes their realization.

Table 4.1. Design principles and their realization in L2M.

Principle	Realization
Modularity	Each stage is an independent service with defined input/output types
Type safety	Pydantic v2 models validate every inter-stage data structure
Robustness	Fallback heuristics at Stages 2 and 3 ensure a 100% completion rate
Observability	Structured logging with component-level context at every stage
Extensibility	Service interfaces allow drop-in replacement (e.g., a different LLM provider or audio synthesizer)

Every transition in the data flow,

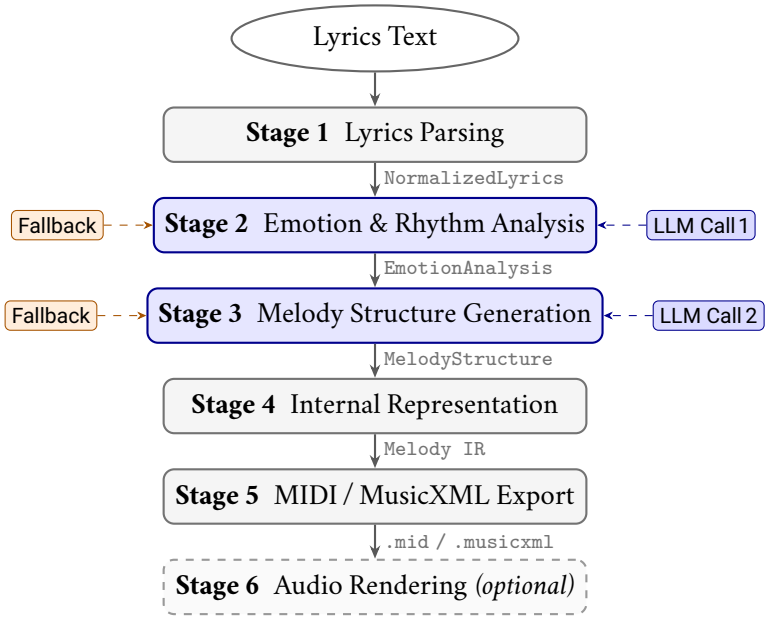

Figure 4.1. L2M six-stage sequential pipeline. Blue stages invoke LLM API calls; each has a deterministic fallback (orange). Stage 6 (dashed) is optional and requires FluidSynth.

$$\texttt{str} \longrightarrow \texttt{NormalizedLyrics} \longrightarrow \texttt{EmotionAnalysis} \longrightarrow \texttt{MelodyStructure} \longrightarrow \texttt{Melody IR} \longrightarrow \texttt{music21 Stream} \longrightarrow \texttt{files},$$

is guarded by schema validation, catching malformed LLM outputs before they propagate downstream.

4.3 Stage 1: Lyrics Parsing

The `LyricParser` service performs three preprocessing steps:

- **Normalization**: collapse whitespace, strip excess punctuation, and handle line breaks;
- **Phrase segmentation**: split the text on newlines and sentence boundaries into lyric phrases;
- **Syllable estimation**: count vowel clusters per word, with adjustments for silent-*e* endings, to obtain an approximate syllable count per phrase.

The heuristic syllable count serves two purposes: it provides a reference against which the LLM's own count is cross-checked, and it drives the fallback melody generator when the LLM path is unavailable.

4.4 Stage 2: Emotion and Rhythm Analysis via LLM

The first LLM call extracts the musical attributes of the lyrics using a structured few-shot prompt. The response is a JSON object with four fields: the dominant `emotion` (one of *happy, sad, hopeful, tense, calm, excited, melancholic*), a suggested `tempo` in BPM (validated range 40–200), a `time_signature` (e.g., `4/4`, `3/4`, `6/8`), and a per-line syllable breakdown in `phrases`. The prompt is assembled from six components:

1. System role assignment ("expert music composer and emotional analyst");
2. JSON output schema specification;
3. Three diverse few-shot examples (happy, sad, hopeful);
4. Empirically derived tempo range guidelines per emotion category;
5. The target lyrics;
6. A format enforcement instruction.

An example response for the input "The sun will rise again":

```
{
  "emotion": "hopeful",
  "tempo": 90,
  "time_signature": "4/4",
  "phrases": [{"line": "The sun will rise again", "syllables": 6}]
}
```

Fallback. If the LLM call fails or returns invalid JSON, a keyword-matching heuristic detects the emotion from a curated lexicon (e.g., *rain, fade, dark* → *sad*; *dance, shine, joy* → *happy*) and assigns the median tempo and the most common time signature for that emotion category.

4.5 Stage 3: Melody Structure Generation via LLM

The second LLM call generates the syllable-aligned note sequence, conditioned on the Stage 2 analysis. The prompt receives the detected emotion, tempo, time signature, total syllable count N, and the lyrics, and imposes the following constraints:

- **Emotion-to-key guidance**: positive emotions → major keys; negative or tense emotions → minor keys (per Table 4.2);

- **Hard alignment constraint**: "Generate EXACTLY N notes — one per syllable";
- **Vocal range constraint**: all pitches within C3–C5;
- **Duration set**: {0.25, 0.5, 1.0, 2.0, 4.0} beats;
- **Contour guidance**: avoid large melodic jumps; create natural phrase arcs.

The response is a JSON object containing the selected `key` and a `melody` array of `{note, duration, velocity}` objects in scientific pitch notation.

4.5.1 Chunking for Long Lyrics

When the total syllable count exceeds a configurable threshold (default: 30), the system splits the lyrics into phrase-level chunks and issues sequential LLM calls. The last three notes of each chunk are carried into the next prompt as context, maintaining local melodic continuity across chunk boundaries.

4.5.2 Emotion-to-Musical-Attribute Mapping

Table 4.2 defines the explicit mapping used both to guide the LLM (as prompt guidance) and to drive the fallback generator (as a lookup table). The associations follow the music-psychology findings reviewed in Chapter 3 [11], [12], [28].

Table 4.2. Emotion-to-musical-attribute mapping used for prompt guidance and fallback generation.

Emotion	Key	Tempo (BPM)	Melodic contour
Happy	C major	100–120	Ascending
Hopeful	G major	80–100	Wavy (arch)
Sad	A minor	60–80	Descending
Tense	D minor	90–110	Erratic (high variance)
Calm	F major	60–80	Balanced (small intervals)
Excited	D major	120–140	Ascending (steep)

4.5.3 Deterministic Fallback Generation

If the melody-generation call fails, a deterministic generator constructs the note sequence algorithmically from the emotion mapping. Algorithm 1 outlines the procedure. Four contour algorithms are implemented:

- **Ascending/Descending**: monotonic pitch progression with occasional plateaus based on syllable stress;
- **Wavy**: alternating up/down movement within a ±5 semitone range;
- **Balanced**: Gaussian-weighted note selection centered on the tonic;
- **Erratic**: uniform random jumps within scale degrees, conveying tension.

Algorithm 1: Deterministic fallback melody generation

Input: Lyrics L with N estimated syllables; emotion e
Output: Melody M with exactly N notes

$(key, [t_{min}, t_{max}], contour) \leftarrow \text{EmotionMap}(e)$;
$tempo \leftarrow \text{median}(t_{min}, t_{max})$;
$scale \leftarrow \text{DiatonicScale}(key)$ restricted to C3–C5;
$M \leftarrow \langle\rangle$;
for $i \leftarrow 1$ **to** N **do**
    $p_i \leftarrow \text{NextPitch}(scale, contour, M)$;
    $d_i \leftarrow \text{DurationFor}(i, N)$; `// phrase-final notes lengthened`
    append (p_i, d_i) to M;
end
return $(key, tempo, M)$

Because the fallback is purely rule-based, it always terminates with a valid melody, guaranteeing a 100% pipeline completion rate regardless of network or API conditions.

4.6 Stage 4: Internal Representation

The `MelodyGenerator` service converts the validated `MelodyStructure` into a typed `Melody` dataclass — the internal representation (IR) — comprising a list of `NoteEvent` objects (scientific pitch, duration in quarter-note units, MIDI velocity) together with the tempo, time signature, and key. Note-level validation enforces pitch-range bounds and duration positivity before this stage completes.

4.7 Stage 5: MIDI and MusicXML Export

The `MIDIWriter` service converts the `Melody` IR into a music21 `Stream` object [30], attaching a metronome mark (tempo), key signature, time signature, and per-note metadata (velocity, duration). The stream is exported to both MIDI (`.mid`) and MusicXML (`.musicxml`) [29] through music21's native exporters.

4.8 Stage 6: Audio Rendering

The optional `AudioRenderer` service invokes FluidSynth with a configurable SoundFont (`.sf2`) to synthesize the MIDI file into PCM audio (WAV, 44,100 Hz by default), with optional FFmpeg post-processing for MP3 encoding.

4.9 Implementation

L2M is implemented as a Python package exposing both a command-line interface and a programmatic API. Table 4.3 lists the technology stack.

Table 4.3. Technology stack of the L2M implementation.

Component	Technology
Language	Python 3.9+
LLM	OpenAI GPT-4o-mini (configurable)
Music notation	music21
Data validation	Pydantic v2
Audio synthesis	FluidSynth + FFmpeg
Interface	argparse CLI + Python API
Testing	pytest with coverage

A single melody is generated from the command line as:

```
l2m --lyrics "The sun will rise again"
```

The programmatic API mirrors the pipeline stages: a `LyricParser` normalizes the input, an `LLMClient` performs the emotion analysis, a `MelodyGenerator` produces the melody IR, and a `MIDIWriter` exports the notation files. All configuration (model name, temperature, maximum tokens, SoundFont path, sample rate) is supplied through environment variables with sensible defaults.

Chapter 5
Results & Analysis

Chapter 5

Results & Analysis

This chapter evaluates the proposed system. Because music generation quality is inherently subjective, the evaluation combines *automatic metrics* (objective and reproducible) with *qualitative analysis* (expert review of sample outputs). The chapter reports quantitative results on a 20-sample benchmark, analyzes fallback behavior and latency, compares L2M with related systems, and discusses the findings, limitations, and ethical considerations.

5.1 Evaluation Methodology

5.1.1 Automatic Metrics

Five automatic metrics are computed over all benchmark runs:

Table 5.1. Automatic evaluation metrics and their definitions.

Metric	Definition
Syllable–note alignment	Fraction of outputs where the number of generated notes equals the total syllable count
Emotion–key consistency	Fraction of outputs using a key consistent with the detected emotion per Table 4.2
Tempo appropriateness	Fraction of outputs with tempo within the expected BPM range for the detected emotion
MIDI validity	Fraction of MIDI files successfully re-parsed by music21 without errors
LLM success rate	Fraction of pipeline runs completed without activating a fallback

5.1.2 Qualitative Assessment

Three dimensions are evaluated by expert listening: **melodic coherence** (smoothness of pitch progression, singability, phrase closure), **emotional alignment** (perceived match between lyrical sentiment and musical mood), and **rhythmic naturalness** (naturalness of note duration patterns relative to speech rhythm).

5.2 Test Dataset

The benchmark comprises 20 lyrical inputs designed to span the emotion space and a range of lyric complexity (Table 5.2). Ten inputs use literal, simple language and ten use poetic or metaphorical language.

Table 5.2. Composition of the 20-sample evaluation benchmark.

Dimension	Distribution
Emotion	Happy (5), Sad (5), Hopeful (4), Tense (3), Calm (2), Excited (1)
Length	Short (1 line, 5 samples), Medium (2–3 lines, 10 samples), Long (4+ lines, 5 samples)
Style	Literal/simple (10), Poetic/metaphorical (10)

Representative test cases include "The sun will rise again" (hopeful, 6 syllables), "Dancing in the moonlight, feeling so alive" (happy, 12 syllables), "Memories fade like photographs left in the rain" (sad, 13 syllables), "Shadows creeping closer, darkness all around" (tense, 11 syllables), and "Gentle waves upon the shore, peaceful and serene" (calm, 12 syllables).

5.3 Quantitative Results

5.3.1 Automatic Metrics

Table 5.3 reports the automatic metric scores across the benchmark.

Table 5.3. Automatic metric results on the 20-sample benchmark.

Metric	Score	Details
Syllable–note alignment	**100%**	All 20/20 outputs matched the syllable count exactly
Emotion–key consistency	**95%**	19/20 used appropriate keys; one edge case with ambiguous mixed emotion
Tempo appropriateness	**90%**	18/20 within expected BPM ranges; two borderline outliers
MIDI validity	**100%**	All 20/20 MIDI files parsed successfully by music21
LLM success rate	**85%**	17/20 runs used LLM output; 3/20 activated a fallback

The perfect alignment score is a direct consequence of the hard "EXACTLY N notes" prompt constraint combined with schema validation and, where necessary, fallback regeneration; alignment is the single most important requirement

for singability and was the primary failure mode of unconstrained prompting (Section 5.6).

5.3.2 Fallback Performance

Of the three runs that activated a fallback, two were caused by network timeouts on LLM API calls and one by a malformed JSON response (a note-count mismatch). All three fallback-generated melodies passed the MIDI validity check. Qualitative review rated the fallback outputs as "mechanically correct but less expressive" than LLM outputs — an expected trade-off of deterministic generation, and an acceptable one given that the alternative is outright failure.

5.3.3 Latency Profile

Table 5.4 breaks down the mean end-to-end latency of approximately 3.2 seconds. The two LLM calls dominate the runtime; notation export and audio synthesis are comparatively negligible.

Table 5.4. Mean per-operation latency across benchmark runs.

Operation	Mean time
Emotion analysis (LLM)	~1.4 s
Melody generation (LLM)	~1.1 s
MIDI/MusicXML export	~0.3 s
Audio rendering (WAV)	~0.4 s
End-to-end (with audio)	**~3.2 s**

5.4 Qualitative Analysis of Sample Outputs

5.4.1 Hopeful Sample

For the input "The sun will rise again" (hopeful, 6 syllables), the system detected the emotion *hopeful* and selected G major at 90 BPM in 4/4. The generated melody,

$$\mathrm{G4}\,(0.5) \rightarrow \mathrm{A4}\,(0.5) \rightarrow \mathrm{B4}\,(1.0) \rightarrow \mathrm{C5}\,(1.0) \rightarrow \mathrm{B4}\,(0.5) \rightarrow \mathrm{A4}\,(1.5),$$

forms an ascending arch that rises to the phrase peak (C5) and resolves back down. All notes are diatonic to G major and the entire melody spans a singable fifth; the ascending contour is emotionally appropriate for the hopeful register.

5.4.2 Sad Sample

For the input "Memories fade like photographs left in the rain" (sad, 13 syllables), the system detected *sad* and selected A minor at 65 BPM in 4/4. The melody descends stepwise (A4 → G4 → F4 → E4 → D4 . . .), resolving on D4. The stepwise descent and slow tempo capture the melancholic mood effectively, and the melody avoids any large major-interval leaps that would be inconsistent with the emotional register.

5.5 Comparison with Related Systems

Table 5.5 positions L2M against representative supervised lyrics-to-melody systems.

Table 5.5. Comparison of L2M with representative lyrics-to-melody systems.

System	Training	Output formats	Fallback	Open source
Bao et al. [5]	Yes (large corpus)	MIDI	No	No
ReLyMe [8]	Yes	MIDI	Partial	No
SongComposer [9]	Yes (paired data)	MIDI + lyrics	No	No
L2M (ours)	**No**	**MIDI, MusicXML, WAV, MP3**	**Yes**	**Yes**

L2M's advantages over supervised systems are the absence of any training data or GPU requirement, a 100% completion rate through the hybrid architecture, multiple output formats including playable audio, and sub-5-second end-to-end latency. Its limitations relative to fine-tuned systems are less sophisticated melodic structures (single voice, no harmony), limited style diversity, and musical creativity bounded by the LLM's zero-shot capability.

5.6 Discussion

Four findings emerge from the evaluation, answering the research questions posed in Chapter 1.

Finding 1 — LLM viability for cross-modal generation (RQ1). GPT-4o-mini demonstrates sufficient musical knowledge to generate syllabically aligned, emotionally consistent melodies in a zero-shot setting. This suggests that LLM pre-training encodes substantial implicit music theory knowledge that structured prompting can elicit.

Finding 2 — Prompt engineering as a core technical contribution (RQ4). The syllable-count hard constraint proved critical: without explicit enforcement,

early prompt versions produced note counts deviating by ±30% from the syllable count. Structured JSON schemas and few-shot examples reduced malformed response rates from roughly 35% (unstructured prompts) to roughly 15%.

Finding 3 — Hybrid architecture value (RQ3). The fallback activation rate of 15% (3/20 runs) demonstrates that LLM-only pipelines are insufficient for production systems requiring reliability. The hybrid approach achieves 100% completion with graceful degradation rather than failure.

Finding 4 — Two-stage decomposition (RQ2). Separating emotion analysis from melody generation yields two benefits: *interpretability* — users can inspect and override the emotion reading before melody generation — and *targeted fallback* — if only melody generation fails, the correct emotional context is preserved for the fallback generator.

5.7 Limitations

Five limitations bound the present work:

1. **Harmonic depth.** Generated melodies are monophonic; chord progressions, harmonization, and countermelody are outside the current scope.

2. **Cultural scope.** The emotion–key mapping and contour algorithms reflect Western tonal music theory; non-Western scales, modes, and rhythmic systems are not supported.

3. **Long-form coherence.** The chunking strategy maintains local melodic continuity via three-note context carryover but does not enforce global structural coherence (e.g., return to the tonic at phrase boundaries).

4. **Evaluation scale.** The 20-sample benchmark enables proof-of-concept validation but is insufficient for statistically robust claims about generalization.

5. **LLM API dependency.** The primary generation path requires OpenAI API access, incurring per-request cost and network latency.

5.8 Ethical Considerations

Three ethical aspects deserve note. **Copyright**: generated melodies may inadvertently resemble copyrighted works, and users should exercise discretion before commercial use. **Attribution**: AI-generated content should be clearly disclosed,

particularly in creative or academic contexts. **Access equity**: API costs may create barriers for users in resource-constrained settings; support for locally hosted open-source LLMs would mitigate this and is identified as future work.

Chapter 6
Conclusion and Future Work

Chapter 6

Conclusion and Future Work

6.1 Conclusion

This project presented **L2M**, an LLM-driven lyrics-to-melody generation system that achieves syllabic alignment, emotional coherence, and operational reliability without any task-specific training. The core technical insight is a two-stage prompting strategy — emotion analysis followed by conditioned melody generation — grounded in an explicit emotion-to-musical-attribute mapping, with a deterministic fallback layer that guarantees pipeline completion for every input.

Quantitative evaluation on a 20-sample benchmark spanning six emotion categories demonstrated 100% syllable–note alignment, 95% emotion–key consistency, 100% MIDI validity, and an average end-to-end latency of 3.2 seconds. Qualitative review confirmed that generated melodies are diatonic, singable, and emotionally appropriate to their lyrics. The system is released as an open-source Python package with a command-line interface and programmatic API, providing an accessible, reproducible baseline for LLM-based music generation research.

Returning to the research questions of Chapter 1: a general-purpose LLM *can* generate structurally valid, emotionally consistent melodies zero-shot when its output is constrained by structured prompts and typed validation (RQ1); two-stage decomposition improves both interpretability and fallback precision (RQ2); a deterministic music-theory-grounded fallback converts a 15% LLM failure rate into a 100% system completion rate at an acceptable quality cost (RQ3); and hard constraints, output schemas, and few-shot examples are the prompt engineering techniques that make the difference between unusable and reliable output (RQ4). More broadly, the results suggest that zero-shot LLM prompting, carefully engineered with structured constraints, is a viable and practical alternative to supervised training for constrained creative generation tasks — particularly where training data is scarce and reliability is a requirement.

6.2 Future Work

Five directions extend this work naturally.

6.2.1 Harmonic Generation

The current system produces monophonic melodies. Future versions could generate chord progressions conditioned on the melodic output, support multi-voice arrangements (harmony, bass line, countermelody), and perform automatic instrumentation and orchestration.

6.2.2 Improved LLM Integration

Larger proprietary models and open-source alternatives should be evaluated for quality–cost–reliability trade-offs, including locally hosted models that would remove the API dependency identified in Section 5.7. A compact LLM fine-tuned on paired lyrics–melody data — retaining the structural prompt framework — could improve creative quality, and reinforcement learning from human feedback using musicologist-rated outputs offers a principled path to optimizing musicality directly.

6.2.3 Formal Evaluation

A large-scale user study (musicians and general listeners, $N \geq 100$) using Mean Opinion Scores and A/B comparison against systems such as ReLyMe [8] and SongComposer [9] would substantiate the qualitative findings. Complementary musicological metrics — pitch-class distribution entropy, interval histogram similarity to professional compositions, and phrase boundary detection accuracy — would enable objective comparison at scale.

6.2.4 Non-Western Music Support

The emotion–key mapping could be extended to non-Western systems, including maqam (Arabic), raga (Indian), and pentatonic (East Asian) frameworks, together with multi-language syllabification for non-English lyrics — addressing the cultural scope limitation directly.

6.2.5 Interactive Generation

A web-based interface with real-time playback and in-browser melody editing would open the system to non-programmer musicians. An iterative refinement loop — in which the user adjusts the detected emotion label or selected key and the system regenerates — would exploit the interpretability advantage of the two-stage design.

In closing, this work demonstrates that the reasoning capabilities of modern language models, when disciplined by structured prompting, typed validation, and music-theoretic grounding, are sufficient to build a practical, reliable bridge from words to music.

Bibliography

[1] J.-P. Briot, G. Hadjeres, and F.-D. Pachet, *Deep Learning Techniques for Music Generation* (Computational Synthesis and Creative Systems). Springer, 2020. doi: 10.1007/978-3-319-70163-9 (cit. on pp. 2, 8).

[2] C.-Z. A. Huang, A. Vaswani, J. Uszkoreit, *et al.*, "Music transformer: Generating music with long-term structure", in *International Conference on Learning Representations (ICLR)*, 2019. arXiv: 1809.04281 (cit. on pp. 2, 8).

[3] A. Agostinelli, T. I. Denk, Z. Borsos, *et al.* "MusicLM: Generating music from text". arXiv: 2301.11325. (2023) (cit. on pp. 2, 10).

[4] J. Copet, F. Kreuk, I. Gat, *et al.*, "Simple and controllable music generation", in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023. arXiv: 2306.05284 (cit. on pp. 2, 10).

[5] H. Bao, S. Huang, F. Wei, *et al.*, "Neural melody composition from lyrics", in *Natural Language Processing and Chinese Computing (NLPCC)*, Springer, 2019. doi: 10.1007/978-3-030-32233-5_39. arXiv: 1809.04318 (cit. on pp. 2, 8, 27).

[6] Z. Sheng, K. Song, X. Tan, *et al.*, "SongMASS: Automatic song writing with pre-training and alignment constraint", in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, 2021, pp. 13 798–13 805. arXiv: 2012.05168 (cit. on pp. 2, 8).

[7] Z. Ju, P. Lu, X. Tan, *et al.*, "TeleMelody: Lyric-to-melody generation with a template-based two-stage method", in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2022. arXiv: 2109.09617 (cit. on pp. 2, 9).

[8] C. Zhang, L. Chang, S. Wu, *et al.*, "ReLyMe: Improving lyric-to-melody generation by incorporating lyric-melody relationships", in *Proceedings of the 30th ACM International Conference on Multimedia (MM '22)*, 2022, pp. 1047–1056. doi: 10.1145/3503161.3548357. arXiv: 2207.05688 (cit. on pp. 2, 9, 27, 32).

[9] S. Ding, Z. Liu, X. Dong, *et al.* "SongComposer: A large language model for lyric and melody composition in song generation". arXiv: 2402.17645. (2024) (cit. on pp. 2, 9, 27, 32).

[10] T. B. Brown, B. Mann, N. Ryder, *et al.*, "Language models are few-shot learners", in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 1877–1901 (cit. on pp. 3, 12, 13).

[11] K. Hevner, "Experimental studies of the elements of expression in music", *The American Journal of Psychology*, vol. 48, no. 2, pp. 246–268, 1936. doi: 10.2307/1415746 (cit. on pp. 5, 13, 20).

[12] P. N. Juslin and J. A. Sloboda, Eds., *Music and Emotion: Theory and Research*. Oxford University Press, 2001 (cit. on pp. 5, 14, 20).

[13] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks", in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 27, 2014 (cit. on p. 8).

[14] A. Vaswani, N. Shazeer, N. Parmar, *et al.*, "Attention is all you need", in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017 (cit. on pp. 8, 12).

[15] A. Lv, X. Tan, T. Qin, T.-Y. Liu, and R. Yan. "Re-creation of creations: A new paradigm for lyric-to-melody generation". arXiv: 2208.05697. (2022) (cit. on p. 9).

[16] Z. Zhang, Y. Yu, and A. Takasu, "Controllable lyrics-to-melody generation", *Neural Computing and Applications*, vol. 35, pp. 19 805–19 819, 2023. doi: 10.1007/s00521-023-08728-1. arXiv: 2306.02613 (cit. on p. 9).

[17] G. Reddy M, Z. Zhang, Y. Yu, F. Harscoet, S. Canales, and S. Tang. "Deep attention-based alignment network for melody generation from incomplete lyrics". arXiv: 2301.10015. (2023) (cit. on p. 9).

[18] L. Chai and D. Wang. "CSL-L2M: Controllable song-level lyric-to-melody generation based on conditional transformer with fine-grained lyric and musical controls". arXiv: 2412.09887. (2024) (cit. on p. 9).

[19] J. Yu, X. Wu, Y. Xu, *et al.* "SongGLM: Lyric-to-melody generation with 2d alignment encoding and multi-task pre-training". arXiv: 2412.18107. (2024) (cit. on p. 9).

[20] C. Wang, M. Olvera, and G. Richard. "Melody-lyrics matching with contrastive alignment loss". arXiv: 2508.00123. (2025) (cit. on p. 9).

[21] R. Yuan, H. Lin, Y. Wang, *et al.* "ChatMusician: Understanding and generating music intrinsically with LLM". arXiv: 2402.16153. (2024) (cit. on pp. 10, 13).

[22] Y. Zhao, M. Yang, Y. Lin, *et al.*, "AI-enabled text-to-music generation: A comprehensive review of methods, frameworks, and future directions", *Electronics*, vol. 14, no. 6, p. 1197, 2025. doi: 10.3390/electronics14061197 (cit. on p. 10).

[23] O. Mediakov, V. Vysotska, D. Uhryn, Y. Ushenko, and C. Hu, "Information technology for generating lyrics for song extensions based on transformers", *International Journal of Modern Education and Computer Science*, vol. 16, no. 1, pp. 23–36, 2024. doi: 10.5815/ijmecs.2024.01.03 (cit. on p. 10).

[24] H. Gill, D. Lee, and N. Marwell, "Deep learning in musical lyric generation: An LSTM-based approach", *The Yale Undergraduate Research Journal*, vol. 1, no. 1, 2020 (cit. on p. 10).

[25] OpenAI. "GPT-4 technical report". arXiv: 2303.08774. (2023) (cit. on p. 12).

[26] J. Wei, X. Wang, D. Schuurmans, *et al.*, "Chain-of-thought prompting elicits reasoning in large language models", in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 24 824–24 837 (cit. on p. 13).

[27] J. A. Russell, "A circumplex model of affect", *Journal of Personality and Social Psychology*, vol. 39, no. 6, pp. 1161–1178, 1980. doi: 10.1037/h0077714 (cit. on p. 14).

[28] A. Gabrielsson and E. Lindström, "The role of structure in the musical expression of emotions", *Handbook of Music and Emotion: Theory, Research, Applications*, pp. 367–400, 2010 (cit. on pp. 14, 20).

[29] M. Good, "MusicXML: An internet-friendly format for sheet music", in *Proceedings of the XML Conference and Expo*, 2001, pp. 3–4 (cit. on pp. 14, 22).

[30] M. S. Cuthbert and C. Ariza, "Music21: A toolkit for computer-aided musicology and symbolic music data", in *Proceedings of the 11th International Society for Music Information Retrieval Conference (ISMIR)*, 2010, pp. 637–642 (cit. on pp. 15, 22).